Screener tool API

Every HTTP route a screener tool answers: 81 routes in 14 route files under `src/routes/`, plus the routes defined in `src/index.js` itself. Every tool answers the same routes on its own port (T1 :3000, T2 :3010 …), against its own database. The only exception is `/api/tools/manage`, which exists on T1 only.

qp (`:8765`) and the journal (`:3100`) have their own APIs, not listed here.

Related: what each feature does (DATA-PIPELINE.md, REGISTERS.md, SETTINGS.md, MONITOR.md, DATACHECK.md, TOOL-LIFECYCLE.md).

Conventions

- **Login.** Every route needs the `td_auth` cookie (the login is documented with the system-wide pages). Requests from the box itself (loopback, no proxy headers) pass without it, which is how tools, qp and the scheduler call each other. Without a cookie a page request is sent to `/login`; an API request gets `401` (`503` while no password is set: locked).
- **Answers** are JSON, usually `{ ok: true, … }` or `{ ok: false, error: "…" }` with `400` (bad input), `404` (not found) or `500` (failure). A few older routes return the data without `ok`.
- **Unknown `/api/…` paths** answer `404 { ok: false, error: "No such endpoint", path }`, never the landing page.
- **Cross-port reads.** The landing page (served by T1) reads the other tools directly; only these paths allow that origin: `/health`, `/api/tools`, `/api/registry/today`, `/api/analysis/status`, `/api/analysis/screener-report`, `/api/shortlist/all-tools`, `/api/shortlist/all-tools/export`, `/api/tool`, `/api/version`.
- Dates are `YYYY-MM-DD`, New York time. Tickers are upper-cased by the route.

Pages and login (`src/index.js`, `src/auth`)

Method	Path	Serves
GET	`/`	the landing page (`home.html`)
GET	`/screeners`	the screener list (`screeners.html`)
GET	`/screeners/compare`	the month-end comparison (`compare.html`)
GET	`/screeners/manage`	Manage tools (`manage.html`); its buttons call T1
GET	`/scanner`, `/scanner/*`	the tool's main page (`index.html`)
GET	anything else that is not `/api`	the landing page
GET / POST	`/login`	the login form; POST checks the password and sets the cookie
GET	`/logout`	clears the cookie, back to `/login`

Tool-wide (`src/index.js`)

Method	Path	Input	Answer
GET	`/health`	—	`ok`, `version`, `tool`, `name`, `configName`, `renamed`, `paused`, `pausedAt`, `pausedReason`, `uptimeSec`, `ts`, `check` (when the tool is worth opening: `from`, `to`, `now`, per screener), `summary`
GET	`/api/version`	—	`ok` plus the commit this process loaded at startup
GET	`/api/tools`	—	`tools` (the merged registry, each with its stage and day count), `apps`, `stages`
POST	`/api/tools/:id/stage`	body `{ stage }`; `auto` removes the override	`{ ok, … }`, or `400`

`/api/scan` — scanning (`scan.js`)

Method	Path	Input	Answer
POST	`/run`	—	runs a full scan now: `rowsProcessed`, `ts`, `paused`, `pausedReason`
GET	`/status`	—	`paused`, `pausedAt`, `pausedReason`, `lastRun`, `lastRowCount`, `running`, `error`, `lastReport`, `lastRefresh`
POST	`/capture-r3`	body `{ date? }`	runs the R3 capture (default today)
GET	`/test`	—	raw TradingView probe: `status`, `totalCount`, `rowCount`
GET	`/health`	—	checks TradingView's answer against the expected column types
GET	`/debug`	—	runs the big-moves scan, returns TradingView's raw answer or error

`/api/registry` — today's live list, r0 (`registry.js`)

Method	Path	Input	Answer
GET	`/`	`?tickers=A,B`	`count`, `rows` for those tickers (all when omitted)
GET	`/today`	—	today's rows, live first, then score high to low
GET	`/all`	—	every row in r0
PUT	`/:ticker/bias`	body `{ bias: "auto" | "long" | "short" }`; no body cycles it	`ticker`, `bias`, and the new `_score` and `_scoreDetails` when re-scored; `404` if not in r0

`/api/card` and `/api/news` (`card.js`, `news.js`)

Method	Path	Input	Answer
GET	`/api/card/:ticker`	`?date=` (default today)	`source: "live"` (today, from r0) or `"frozen"` (from R1), `date`, `card`; `404` says whether the ticker was never on this tool or not on that date, with its last date
GET	`/api/news/:ticker`	—	fetches all news sources now, stores it on the card: `ticker`, `news`, `catalyst`

`/api/screeners` — the screener builder (`screeners.js`)

Method	Path	Input	Answer
GET	`/meta`	—	the fields and operators the builder may offer (the validator's own lists)
GET	`/`	—	`screeners`: every screener on this tool
GET	`/history`	—	`totalDays`, `history`: what each screener found, day by day
POST	`/test`	body: a screener definition	runs it unsaved: totals and a sample; `400` lists what is invalid
POST	`/`	body: a definition	creates it: `screener`
PUT	`/:id`	body: the fields to change	partial update: `screener`
PUT	`/:id/name`	body `{ name }`	renames it; its key (stamped on every card) does not change
POST	`/:id/pause`	body `{ reason? }`	stops it adding cards; its cards and history stay
POST	`/:id/resume`	—	—
POST	`/:id/mirror`	—	stores its opposite-facing twin
POST	`/preview-mirror`	body: a definition	what the mirror would be, unsaved: `definition`
DELETE	`/:id`	—	`{ ok }`, `404` if none

`/api/shortlist` (`shortlist.js`)

Method	Path	Input	Answer
POST	`/toggle/:ticker`	body `{ date? }`	stars or un-stars it
GET	`/today`	—	today's shortlist
GET	`/all`	—	every day's shortlist
GET	`/export/:date`	—	a `.txt` download; `404` if none that day
POST	`/run-rule`	—	runs the auto-rule now: `entry`, or `ok: false` with the reason
GET	`/all-tools`	`?date=`	`rows`: the union of every tool's shortlist
GET	`/all-tools/export`	`?date=`, `?format=csv`	the union as a symbol list (one per line, or comma-separated)

`/api/warehouse` — the registers (`warehouse.js`)

Registers: `R0`, `R1`, `R2`, `R3A`, `R3B`, `R4A`, `R4B`, `Shortlist` (REGISTERS.md). Anything else: `400 Invalid register`.

Method	Path	Input	Answer
GET	`/available-dates`	`?register=`	the dates that register holds
GET	`/export/all`	—	every register's data with no date given (R0 = today's live list), keyed by register
GET	`/:register/latest`	—	its most recent day; `404` if empty
GET	`/:register/:date`	—	that day; `404` if none
GET	`/export/:register/:date`	`?format=csv|json` (default csv), `?save=true`	the data; with `save=true`, written as CSV under the tool's temp folder for the scorer and `{ ok, path, rows }` returned
POST	`/import`	body `{ register, date, rows[] }`	**not supported**: `501`, restore from a backup instead
POST	`/collect`	—	runs the R1 and R2 captures now

These read routes are also what the archive answers for a stopped tool (TOOL-LIFECYCLE.md).

`/api/market` — the Market tab (`market.js`)

Method	Path	Input	Answer
GET	`/snapshot`	—	the latest short-term market read (built now if there is none yet); `503` if it cannot be built
GET	`/oneil`	—	O'Neil's market model from the file qp publishes; missing is a normal answer (`ok: false`)
GET	`/groups`	—	group ranks and rank-in-group, plus the industry map's size; asks qp once when the file is missing
GET	`/oneil/stocks`	`?symbols=A,B`	what each stock did on the index's distribution days
GET	`/oneil/13f`	—	the I letter (institutional holders) for every ticker
GET	`/oneil/ratings`	`?symbols=`	U/D volume ratio, accumulation/distribution, RS line and divergence: `stocks` , `fetchedAt`
GET	`/oneil/canslim`	`?symbols=` , `?parts=` , `?cards=`	everything the CANSLIM fold needs (C and A tables, the weekly base) in one call
GET	`/short-interest`	`?symbols=` , `?float=`	a probe of which short-interest source answers, per symbol
POST	`/oneil/seed-industries`	—	rebuilds the industry map from this tool's frozen registers

Details: MARKET-TAB.md.

`/api/analysis` — the scorer (`analysis.js`)

Method	Path	Input	Answer
GET	`/status`	—	`ready`, `scorer`; `ok: false` when the scorer is not reachable
GET	`/model-info`	—	the factor makeup of each table, for explaining a score
POST	`/train`	—	rebuilds the training CSVs from the stored R4 rows and trains: counts, `warnings`
POST	`/score`	body: a card	scores one card (testing)
POST	`/upload-csv`	file, `?register=r4a|r4b`, `?mode=`	loads a CSV into the training table; rows for the same date and ticker are replaced
GET	`/screener-report`	`?entry=A|B`	which screeners find movers (IDEAS.md part 3)
GET	`/training-summary`	—	how much training data there is
DELETE	`/training-data`	`?register=r4a|r4b`, `?date=`	clears that register's rows, or one date's
GET	`/table-data`	`?base=B1…B6`, `?table=`	factor importance and every factor's buckets for one table
GET	`/available-tables`	—	every trained base and its tables

`/api/settings` (`settings.js`)

Method	Path	Input	Answer
GET	`/`	—	`settings`: every known key; secrets as `set` or empty
GET	`/:key`	—	`{ key, value }`; `404` for an unknown key
POST	`/`	body `{ key: value, … }`	`saved: [keys]`, or `400 { errors }` and nothing saved
GET	`/test/:service`	`finnhub`, `alpaca` or `ai`	one live call with the stored key: `ok`, `message`
POST	`/reset-hot`	—	clears the hot-sector state

Every key: SETTINGS.md.

`/api/monitor` (`monitor.js`)

Method	Path	Input	Answer
GET	`/`	—	`pipeline` (as `/api/scan/status`), `jobs`
POST	`/jobs/:jobId/toggle`	—	`jobId`, `enabled`; `404` if no such job
POST	`/jobs/:jobId/schedule`	body `{ schedule }` (cron)	`400` when missing or invalid
POST	`/jobs/:jobId/reset`	—	back to the default schedule

Details: MONITOR.md.

`/api/datacheck` (`datacheck.js`)

Method	Path	Input	Answer
GET	`/`	`?symbol=` (default SPY)	every source judged: `ok`, `total`, `passed`, `degraded`, `down`, `qpReachable`, `checks`, `summary`. Takes up to two minutes

Details: DATACHECK.md.

`/api/tool` — this tool's identity (`tool.js`)

Method	Path	Input	Answer
GET	`/`	—	name, default name, renamed, paused, since when, why
PUT	`/name`	body `{ name }`; the default name clears the override	the identity
POST	`/pause`	body `{ reason? }`	stops new scans; nothing collected is touched
POST	`/resume`	—	the identity

`/api/tools/manage` — Manage tools, T1 only (`toolsManage.js`)

Method	Path	Input	Answer
GET	`/`	—	every tool (deleted ones included) with process state, paused, memory, restarts; the box's free memory; the minimum to start one
POST	`/`	body `{ name, desc?, accent?, captureAt? }`	adds and starts a tool: `tool`, `started`, `note`, `message`
PUT	`/:id`	body: any of `name`, `desc`, `accent`, `captureAt`	`tool`, `message` (says when new times apply)
POST	`/:id/start`	—	`message`
POST	`/:id/stop`	—	not T1
POST	`/:id/pause`	—	calls the tool's `/api/tool/pause`
POST	`/:id/resume`	—	calls the tool's `/api/tool/resume`
POST	`/:id/scan`	—	calls the tool's `/api/scan/run`
POST	`/:id/restore`	—	back on the list, stopped
DELETE	`/:id`	—	not T1; stops it and hides it, history kept

Errors answer `400 { ok: false, error }`. Details: TOOL-LIFECYCLE.md.

Count

File	Routes
analysis.js	10
screeners.js	12
toolsManage.js	10
market.js	9
shortlist.js	7
warehouse.js	7
scan.js	6
settings.js	5
monitor.js	4
registry.js	4
tool.js	4
card.js, datacheck.js, news.js	1 each
Total	**81**

Plus `src/index.js`: 4 API routes and 7 page routes; and the login routes.